Design, Modeling, and Verification of a PDSCH-Oriented 5G NR 64 x 8 Massive MIMO-OFDM Simulink Testbench

Md Moklesur Rahman

Independent Researcher, Finland | moklesur.eee@gmail.com | github.com/dipucwc

Abstract

This report presents the system modeling, execution control, instrumentation, and verification of a PDSCH-oriented 5G NR Massive MIMO-OFDM Simulink testbench. The executed profile contains 64 transmit antennas, 8 receive antennas, four spatial layers, a 12-resource-block reduced OFDM allocation, 16-QAM modulation, wideband SVD eigenbeamforming, DM-RS-based least-squares estimation of the effective channel, and unbiased minimum mean-square error equalization. The mathematical equations in this report are not Simulink-specific formulas. They are the implementation-independent physical-layer and receiver equations executed by the verified MATLAB functions; the Simulink MATLAB Function blocks call those functions through the sl_*.m wrappers and provide scheduling, fixed-size interfaces, signal routing, logging, and instrumentation. The model is generated programmatically from the locked MATLAB configuration, and one complete Monte Carlo frame is executed per fixed simulation step. Before the first frame, the model runs constellation-power, modulation round-trip, OFDM round-trip, and CRC24A verification gates. During execution, the testbench logs BER, BLER, EVM, NMSE, and capacity and provides a passive CP-OFDM waveform-monitoring branch for spectrum, PAPR, and constellation analysis. A seven-point SNR sweep is compared against stored MATLAB reference results using explicit metric-specific tolerances. The supplied result file records PASS at all seven operating points, with a largest BER deviation of 0.063 decades and a largest capacity deviation below 0.5 percent. The project therefore demonstrates a traceable MATLAB-Simulink workflow in which the mathematical specification, numerical source functions, graphical model, result storage, and RF-oriented visualization are linked to one reproducible configuration.

Keywords: 5G NR, PDSCH, Massive MIMO, OFDM, Simulink testbench, wideband SVD, DM-RS, channel estimation, MMSE equalization, verification.

1. Introduction

This section introduces the engineering motivation for the Simulink testbench, defines the scope of the work, states the project objectives, and explains how the remainder of the report is organized.

1.1 Background and motivation

Link-level simulation is used to evaluate the complete physical-layer chain before implementation on radio hardware. In a 5G NR downlink, the PDSCH data path includes transport-block protection, scrambling, QAM mapping, layer mapping, DM-RS insertion, precoding, MIMO channel propagation, channel estimation, equalization, detection, CRC evaluation, and performance measurement [1]-[3]. A MATLAB script can execute these functions sequentially, but a Simulink testbench adds an explicit block-level representation of signal interfaces, execution order, fixed dimensions, logging, and measurement instrumentation [11].

The principal difficulty is not drawing the blocks; it is preserving numerical equivalence between the graphical model and the verified MATLAB functions. A second implementation of the algorithms inside Simulink would introduce another source of mapping, scaling, and indexing defects. The present design therefore uses thin MATLAB Function wrappers around the existing project functions. Simulink controls the schedule and data flow, whereas the numerical operations remain in the verified MATLAB implementation. This arrangement makes the testbench suitable for system integration, model inspection, regression execution, and result logging without creating a divergent algorithm branch.

1.2 Objectives and contributions

The project has four objectives. First, it builds a 64 x 8, four-layer Massive MIMO-OFDM testbench directly from the locked configuration so every fixed-size Simulink interface matches the active profile. Second, it executes one complete Monte Carlo frame per fixed step and produces the same frame metrics as the MATLAB reference. Third, it prevents unverified execution by running four mandatory acceptance gates in the model initialization callback. Fourth, it adds an automated SNR sweep and passive RF-monitoring branch so the same model supports numerical validation and waveform inspection.

1.3 Report organization

Section 2 presents the implementation-independent physical-layer mathematical model and identifies the Simulink subsystem and MATLAB function that execute each equation. Section 3 describes the graphical architecture and the relationship between the blocks, wrapper functions, and core MATLAB functions. Section 4 presents the receiver algorithms and explains how the Simulink estimator and equalizer subsystems execute them. Section 5 explains programmatic model construction and execution control. Section 6 defines the verification methodology. Section 7 reports the executed results and RF measurements. Section 8 provides the reproduction workflow and project contents. Section 9 states the limitations and future work, and Section 10 concludes the report.

2. Physical-Layer Mathematical Model and Simulink Realization

This section presents the implementation-independent mathematical specification of the wireless link and then explains its realization in the Simulink testbench. Equations (1)-(4) are standard complex-baseband MIMO-OFDM, effective-channel, power-normalization, and eigenbeamforming relations; they were not created as special equations for Simulink. The core MATLAB functions perform the numerical operations, while the Simulink MATLAB Function blocks call those functions through the sl_*.m wrappers. Simulink therefore defines the execution order, signal interfaces, fixed dimensions, logging, and instrumentation, whereas the equations define the physical-layer calculation performed on the signals. This separation preserves traceability between the mathematical model, the MATLAB source, the graphical block diagram, and the reported results.

2.1 Per-resource-element MIMO-OFDM model

On active subcarrier k and OFDM symbol m, the four-layer transmit vector s[k,m] is mapped to the 64 transmit antennas by the precoder W and propagated through the 8 x 64 channel matrix H[k,m]. The received vector is

$$y[k,m] = H[k,m]\, W\, s[k,m] + n[k,m] \quad (1)$$

where n[k,m] is complex circular Gaussian noise. The active massive profile uses a flat Rayleigh realization that is constant over the 144 active subcarriers within one frame and changes independently from frame to frame. Equation (1) is evaluated by apply_mimo_channel.m and is reached from the graphical model through the sl_apply_channel.m wrapper. The Simulink MIMO Channel + AWGN subsystem therefore schedules the verified MATLAB calculation rather than defining a different channel equation. Because the DM-RS symbols are precoded with the data, the receiver estimates the effective layer channel

$$G[k,m] = H[k,m]\, W \quad (2)$$

of dimension 8 x 4 rather than the full 8 x 64 physical channel. Equation (2) is the target quantity of estimate_effective_channel_ls.m, called by sl_estimator.m, and it is the channel used by equalize_mimo.m through sl_equalizer.m. The same true effective channel is used by the metric path for the NMSE calculation.

2.2 Power normalization and total-transmit-SNR convention

All QAM constellations are normalized to unit average power per layer, and the columns of the wideband SVD precoder are orthonormal. The total transmitted power is therefore equal to the number of layers L. The noise variance applied per receive antenna is

$$sigma\_n^2 = L / 10^{(SNR\_dB/10)} \quad (3)$$

which is implemented in apply_mimo_channel.m and called from the Simulink model through sl_apply_channel.m. This convention prevents an antenna-count change from being hidden by an unintended transmit-power increase and keeps the MATLAB and Simulink operating points comparable.

2.3 Wideband SVD precoding

The wideband precoder is obtained from the average transmit covariance over the K active subcarriers. The implementation forms

$$R = (1/K) \sum_k H[k]^H H[k], \quad W = eig\_L(R) \quad (4)$$

where eig_L selects the L dominant eigenvectors after numerical Hermitian symmetrization. Equation (4) is implemented by compute_precoder.m and called by the Simulink Wideband SVD Precoder subsystem through sl_precoder.m. One 64 x 4 matrix is reused over all subcarriers. The design follows the eigenbeamforming principle for multi-antenna channels [6], [7] and keeps the effective channel smooth in frequency when a frequency-selective profile is selected.

2.4 Executed configuration

Table 1 summarizes the configuration stored in config.m and in matlab_results_massive_config.txt. The first column groups the parameters by function, the second identifies the parameter, and the third records the value used by the executed 64 x 8 campaign. The table shows that the testbench is a reduced-grid link-level model rather than a full-bandwidth gNB waveform generator.

Table 1. Executed configuration of the 64 x 8 Simulink testbench

Category	Parameter	Executed value
Run control	Random seed	7
Run control	SNR grid	-10, -5, 0, 5, 10, 15, 20 dB
Run control	Frames per SNR point	60
OFDM	FFT / active subcarriers	256 / 144
OFDM	OFDM symbols / slot	14 / 0.5 ms
OFDM	Subcarrier spacing	30 kHz
MIMO	Transmit / receive antennas	64 / 8
MIMO	Spatial layers	4
MIMO	Precoder	Wideband SVD
PDSCH-oriented grid	Modulation / CRC	16-QAM / CRC24A
PDSCH-oriented grid	DM-RS symbols / spacing	4 and 11 / comb 4
Channel and receiver	Channel	Flat Rayleigh
Channel and receiver	Estimator / equalizer	LS / unbiased MMSE

.

The reduced allocation contains 27,624 payload bits per frame after the CRC and resource-grid accounting used by build_frame.m. At 60 frames per SNR point, each operating point evaluates approximately 1.66 million payload bits. This sample size is adequate for regression comparison in the moderate-BER region, but zero observed errors at high SNR must be interpreted as finite-run observations rather than exact zero error probability.

3. Simulink Testbench Architecture

This section describes the graphical architecture, the signal interfaces, the fixed-step execution model, the passive waveform branch, and the traceability from every Simulink subsystem to the MATLAB function that performs its numerical operation.

3.1 Graphical system organization

Figure 1 illustrates the complete testbench. The model is divided into five labeled areas: transmitter, channel and precoding, receiver, metrics and logging, and passive waveform monitoring. The Frame Clock drives both the transmitter and channel generator so one new PDSCH frame and one new channel realization are produced at each simulation step. The SNR Constant block supplies the same total-transmit-SNR value to the channel and metric blocks. The received grid is sent to both the LS estimator and the equalizer, while the metric block also receives the transmitted layer grid, CRC-protected bits, true channel, and precoder so every performance quantity is calculated from consistent frame data.

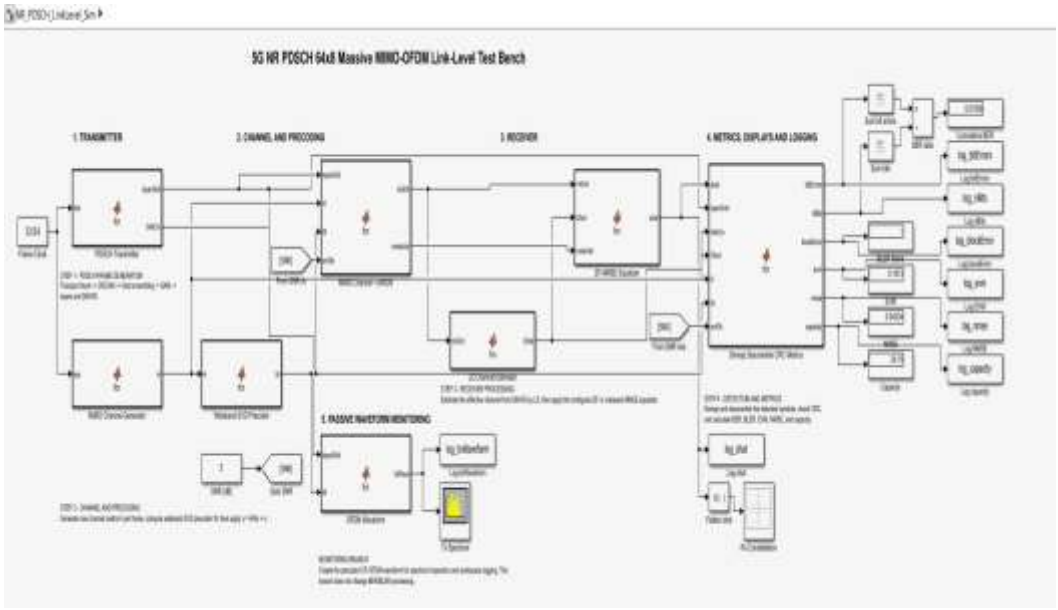

Figure 1. Original 64 x 8 Massive MIMO-OFDM Simulink testbench, executing one Monte Carlo frame per simulation ste

The transmitter block is positioned at the left of Figure 1, the channel and wideband precoder are in the center-left, and the receiver occupies the center-right. The original model colors, MATLAB Function icons, scope icon, display values, and signal routes are retained so the figure matches the executable SLX model. The metric block produces bit errors, evaluated bits, block error, EVM, NMSE, and capacity. Each scalar metric is displayed for the current frame and stored in a To Workspace array. A cumulative BER path independently integrates the bit-error and bit-count streams for live observation; the exact aggregate BER reported after the run is calculated from sum(log_bitErrors)/sum(log_nBits)

3.2 Block-to-function traceability

Table 2 describes the implementation traceability. Each row identifies one Simulink block, its wrapper function, and the MATLAB function that performs the algorithm. The table is important because it separates system-model responsibilities from numerical-algorithm responsibilities: the block defines the interface and schedule, the wrapper loads the active configuration and fixed layout, and the core function performs the verified computation.

Table 2. Simulink block-to-MATLAB-function traceability.

Simulink block	Wrapper	Core MATLAB implementation
PDSCH Transmitter	sl_transmitter.m	build_frame.m and CRC, scrambling, QAM, DM-RS functions
MIMO Channel Generator	sl_channel.m	generate_channel.m
Wideband SVD Precoder	sl_precoder.m	compute_precoder.m
MIMO Channel + AWGN	sl_apply_channel.m	apply_mimo_channel.m
LS Channel Estimator	sl_estimator.m	estimate_effective_channel_ls.m
ZF-MMSE Equalizer	sl_equalizer.m	equalize_mimo.m
Demap Descramble CRC Metrics	sl_metrics.m	compute_frame_metrics.m
OFDM Waveform	sl_waveform.m	ofdm_modulate.m

The wrappers use coder.extrinsic and preallocated fixed-size outputs. Consequently, the model runs the MATLAB algorithms in interpreted mode and does not require a selected C compiler. This choice is appropriate for a validation testbench, but it also means the model is not prepared for code generation without replacing the extrinsic calls with code-generation-compatible implementations.

3.3 Fixed-step execution and logging

The model uses the FixedStepDiscrete solver with a fixed step of one. A simulation stop time of 59 executes 60 frames because the frame index runs from 0 through 59. The initialization callback adds the project tree to the path, loads the active configuration, restores the fixed random seed, and executes the verification gates. The stop callback calculates aggregate BER, BLER, mean EVM, mean NMSE, and mean capacity from the workspace logs. These callbacks make the model self-checking and self-reporting rather than relying on manual post-processing.

4. Receiver Algorithms and Simulink Realization

This section presents the step-by-step frame procedure scheduled by Simulink and then derives the receiver algorithms executed inside that schedule. Equations (5) and (6) are standard channel-estimation and linear-equalization equations [5], [6], [8]-[10]; they are not equations designed only for the Simulink software environment. The verified MATLAB functions estimate_effective_channel_ls.m and equalize_mimo.m implement the numerical algorithms, and the Simulink subsystems call them through sl_estimator.m and sl_equalizer.m. Simulink defines the block interfaces, execution order, fixed dimensions, logging, and instrumentation, whereas the equations define the numerical operations performed on the received signals. The workflow is shown both as a diagram and as a numbered algorithm so that every processing stage can be traced from the graphical model to the MATLAB source.

4.1 One-frame execution procedure

Figure 2 illustrates the algorithm workflow of one Simulink step. The workflow begins with the locked configuration and ends with the workspace logs. Every box corresponds to one block in Figure 1 and to one wrapper listed in Table 2. The algorithm workflow procedure is therefore a compact traceability map between the graphical model and the MATLAB source.

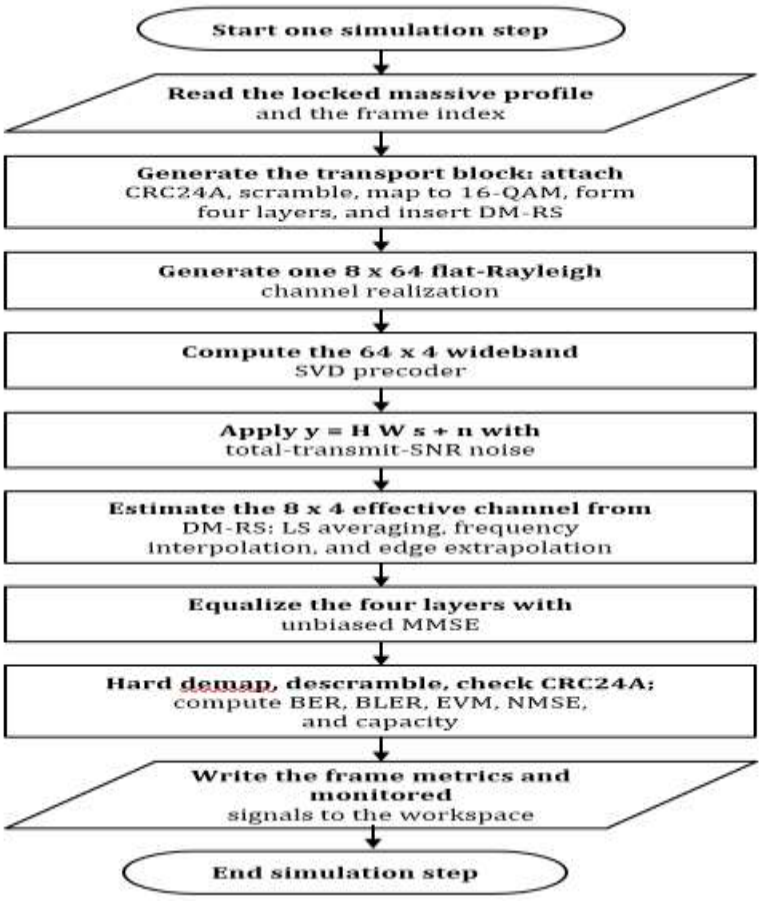

Figure 2. Algorithm workflow of one 64 x 8 Massive MIMO Simulink frame.

Algorithm 1: One-frame 5G NR Massive MIMO-OFDM Simulink execution
1: Input the frame index and total-transmit-SNR value; load the massive profile from config.m.
2: Generate the transport block, attach CRC24A, scramble with the NR Gold sequence, map to unit-power 16-QAM, arrange four layers, and insert the layer DM-RS.
3: Generate one flat-Rayleigh channel H of dimension 144 x 8 x 64.
4: Form the average transmit covariance and obtain the dominant four eigenvectors as the 64 x 4 wideband SVD precoder W.
5: Apply the precoder, MIMO channel, and calibrated complex Gaussian noise to obtain the received grid and noise variance.
6: Estimate the 8 x 4 effective channel from the two DM-RS symbols by LS division, averaging, frequency interpolation, and linear edge extrapolation.
7: Apply the configured unbiased-MMSE equalizer to every data resource element.
8: Hard demap, descramble, evaluate CRC24A, and calculate frame BER, BLER, EVM, NMSE, and capacity.
9: Write the frame metrics, equalized symbols, and monitored waveform to the MATLAB workspace.

The uploaded implementation uses linear interpolation with linear extrapolation at the outer subcarriers in estimate_effective_channel_ls.m. For the executed flat-Rayleigh profile, the physical channel is constant across frequency, so interpolation bias is absent and the estimation error is dominated by pilot noise. If the same testbench is extended to strongly frequency-selective channels, the edge rule should be treated as a controlled design parameter and verified separately [8]-[10].

4.2 DM-RS-based effective-channel estimation

At a DM-RS resource element, the known pilot r[k,m] permits the least-squares estimate

$$G\_hat\_LS[k,m] = Y\_DMRS[k,m] / r[k,m] \quad (5)$$

The implementation averages the observations from the two DM-RS symbols at each pilot subcarrier, reducing the variance of independent pilot noise. It then interpolates the real and imaginary parts separately across frequency. In the supplied source, estimate_effective_channel_ls.m uses linear interpolation with linear extrapolation at the allocation edges. The sl_estimator.m wrapper reconstructs the deterministic DM-RS layout and calls this core function; it does not contain a separate estimation formula. The resulting estimate has dimensions 144 x 8 x 4 and is supplied to both the equalizer and the NMSE calculation.

4.3 Unbiased-MMSE equalization

For each subcarrier, the raw MMSE equalizer is

$$F\_MMSE = (G\_hat^H G\_hat + sigma\_n^2 I)^(-1) G\_hat^H \quad (6)$$

The regularization term limits noise enhancement when the channel matrix is poorly conditioned. Equation (6) is evaluated by equalize_mimo.m, which is called by the Simulink ZF-MMSE Equalizer subsystem through sl_equalizer.m. Because the raw MMSE estimate is amplitude-biased, equalize_mimo.m divides each layer output by the corresponding diagonal element of F_MMSE G_hat. This normalization restores the 16-QAM decision geometry and is essential for correct hard detection of the inner and outer constellation amplitudes. The wrapper controls the interface and deterministic data-position layout but does not redefine the equalizer.

4.4 Performance metrics

The metric block computes BER and BLER from the recovered bit stream and CRC verdict, EVM from the error between equalized and transmitted QAM symbols, NMSE from the estimated and true effective channels, and a layer-domain capacity reference from the true effective channel. Capacity is a theoretical reference in bit/s/Hz and is not interpreted as achieved uncoded throughput [7].

4.5 Cyclic-prefix timing and carrier-frequency-offset synchronization

The advanced testbench extends the receiver with the two synchronization functions that the baseline bench assumes ideal. Both estimates come from the redundancy of the cyclic prefix: every OFDM symbol repeats its last cpLen samples one FFT length earlier, so the received waveform correlates with itself at a lag of exactly nFFT samples wherever the symbol windows are

correctly aligned. For every candidate integer offset d the synchronizer accumulates, over all fourteen symbols and all receive antennas, the correlation C(d) between each prefix window and its repetition, normalizes the magnitude by the average energy of the two windows, and selects the offset that maximizes the normalized metric. This is the classical maximum-likelihood timing estimator for CP-OFDM in white noise, and the normalization makes the metric independent of the received power.

The same winning correlation also carries the frequency estimate. A carrier frequency offset of epsilon subcarrier spacings advances the carrier phase by exactly 2 pi epsilon across one FFT length, so the phase of C(d_hat) divided by 2 pi is the maximum-likelihood estimate of the fractional offset, unambiguous over plus or minus half a subcarrier spacing. The receiver de-rotates the waveform with the estimated offset, advances it to the estimated start, and passes it to the unitary CP-OFDM demodulator of the verified chain; the constant phase term that remains after de-rotation is absorbed by the DM-RS least-squares estimator, which estimates the effective channel including any common rotation. The estimator therefore requires no modification of the verified receiver stages that follow it.

The realization is one wrapper function per stage: sl_waveform_tx precodes the layer grid and produces the CP-OFDM antenna waveform with the verified modulator, sl_channel_impairments propagates the waveform through the flat massive channel in the time domain and injects the configured offsets together with receiver noise at the fixed total-transmit-SNR convention, and sl_synchronizer performs the search, the correction, and the demodulation. The time-domain flat-channel product is exactly equivalent to the per-subcarrier frequency-domain application of the verified chain, and the impairment wrapper refuses frequency-selective models so that the equivalence can never be silently violated.

4.6 Impairment modeling: fractional timing and oscillator phase noise

Beyond the integer offset and the frequency offset, the impairment wrapper models two further effects. A fractional part of the timing offset is applied as an ideal all-band delay through a frequency-domain phase ramp; after the synchronizer recovers the integer part, the fractional residual reaches the receiver as a linear phase across the subcarriers, which the interpolating least-squares estimator absorbs exactly because it estimates the effective channel per subcarrier. Oscillator phase noise is modeled as a Wiener process whose increment variance per sample equals 2 pi times the configured linewidth divided by the sample rate, the standard free-running-oscillator model; the linewidth is set in config.m and disabled by default so that the baseline operating point is unchanged. The DM-RS estimator absorbs the average common phase of the two pilot symbols, while the symbol-to-symbol phase walk between them remains as an error-vector contribution that grows with the linewidth, which is exactly the behavior the executed gates measure.

5. Programmatic Model Construction and Instrumentation

This section explains how the model is built, how the active profile controls all dimensions, how initialization and stop callbacks are installed, and how the passive RF-monitoring branch is added without changing the communication metrics.

5.1 Dimension-derived model generation

build_nr_pdsch_simulink.m first loads config.m and probes build_frame.m while restoring the random state. From the probe it obtains the number of data positions and CRC-protected bits; from the configuration it obtains the OFDM, antenna, and layer dimensions. The script creates a new model, installs the fixed-step solver, creates the MATLAB Function blocks, declares fixed complex output arrays, wires the signal routes, adds the metric displays and workspace sinks, installs annotations, saves the model, and opens it in the Simulink editor.

build_massive_testbench.m is the profile-specific entry point. It backs up the compact 4 x 4 model, calls set_sim_profile(massive), rebuilds every MATLAB Function block with 64 x 8 and four-layer dimensions, and finally calls add_rf_measurements.m. This rebuild is mandatory whenever nTx, nRx, nLayers, nSC, nSymbols, modulation order, or DM-RS allocation changes because these quantities determine fixed block interfaces.

5.2 Passive RF-monitoring branch

The monitoring branch receives the transmitted layer grid and wideband precoder, reconstructs the 64 antenna-domain grid, and applies unitary CP-OFDM modulation. It writes log_txWaveform and log_shat to the workspace and, when the required products are licensed, attaches a live Spectrum Analyzer and Constellation Diagram. The branch consumes no additional random numbers and has no feedback connection to the receiver; therefore, adding or removing the scopes does not change BER, BLER, EVM, NMSE, or capacity.

5.3 Advanced hierarchical testbench with impairment injection and synchronization

The project deliberately maintains two Simulink models. The baseline bench NR_PDSCH_LinkLevel_Sim.slx of Sections 3 and 5.1 remains the verified reference that produced every executed result of Section 7 and is never modified; the advanced bench NR_PDSCH_Advanced_Testbench.slx described here is a separate model that extends the identical verified chain with impairment modeling, synchronization, hierarchical structure, and additional measurement, so the reference and the extension can always be compared side by side. This reference-plus-extension structure follows standard model-based design practice: requirements are validated on the locked reference model first, and every added capability is introduced in the extended model with its own gates.

The build_advanced_testbench script constructs the advanced model NR_PDSCH_Advanced_Testbench.slx from the same proven block classes and wiring helpers as the baseline builder and then folds the wired stages into five named subsystems that mirror the physical structure of the link: 01 Transmitter, 02 Channel and Impairments, 03 Synchronization, 04 Receiver, and 05 Metrics and Verification. The subsystem ports are generated by the Simulink subsystem-creation service directly from the existing connectivity, so the hierarchical organization adds no hand-drawn connection that could deviate from the verified flat chain. Two annotated top-level constants set the impairment operating point, a carrier frequency offset of 0.15 subcarrier spacings and a timing offset of 7 samples, matching the executed synchronization gates, and the SNR constant is distributed with globally scoped tags so that one value drives the channel and the metric stages.

The measurement side grows accordingly. The synchronizer exposes its timing and frequency estimates on dedicated displays and workspace logs, a Link Quality block derives the per-layer post-equalization SINR and the CRC-gated payload throughput from quantities the verified chain already produces, and the cumulative BER path, the frame metric displays, and the complete workspace logging of the baseline bench are retained unchanged. The passive RF branch attaches the licensed Spectrum Analyzer to the transmit waveform at the compact-grid sample rate and the Constellation Diagram to the equalized symbols, with permanent workspace logs of both signals for license-free post-processing. The chain verification gates and the synchronization gates both execute in the model InitFcn, so the advanced model can never produce results over a broken chain or a broken synchronizer. Three further design measures make the bench self-checking and campaign-capable: a Runtime Checks block compares the synchronizer estimates with the injected impairment values on every simulation step within tolerances matched to the Monte Carlo operating range, drives a warning-mode Assertion block, and logs the per-frame decision so that the sweep reports a measured synchronization hit rate per SNR point instead of hiding estimator degradation or aborting a campaign on a single low-SNR miss, the key signals are named on the diagram for readable tracing and selective logging, and every processing block carries a browser description of its function. The run_advanced_sweep runner completes the instrument: it sets the SNR constant across the configured grid, executes the model once per point with the gates re-armed in every InitFcn, collects the

cumulative statistics from the workspace logs, and writes one CSV row per point next to the scripted MATLAB and Python reference curves for direct comparison under the identical impairment operating point.

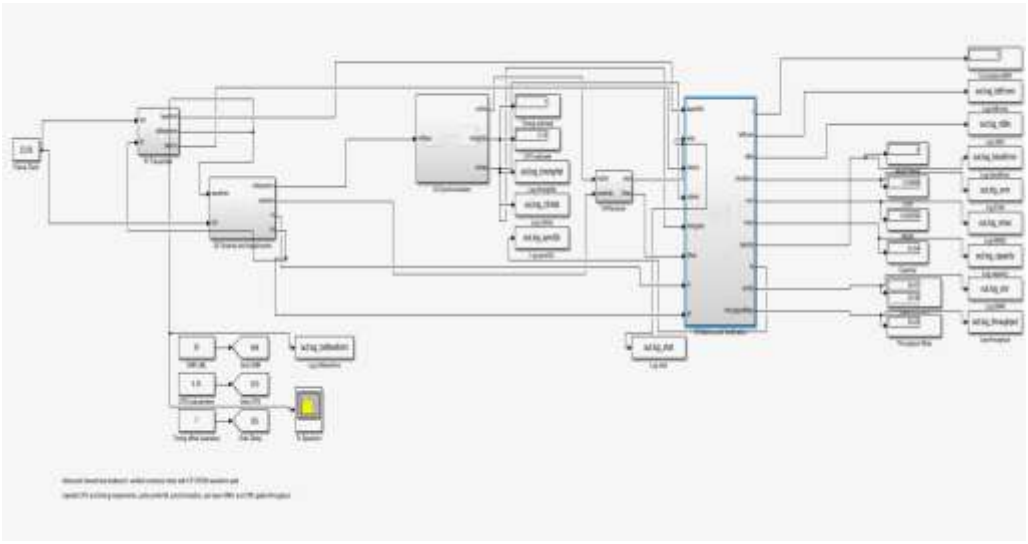

Figure 8. Executed advanced hierarchical testbench at the 20 dB operating point with the injected 0.15-subcarrier carrier frequency offset and 7-sample timing offset.

Figure 8 shows the advanced model after a complete sixty-frame run on the massive profile. The five subsystems carry the labeled signal path from the transmitter through the impairment channel, the synchronizer, and the receiver to the metric stage, and the displays record the executed results of the run: the timing estimate reads exactly 7 samples and the frequency estimate exactly 0.15 subcarrier spacings, matching the injected values, the cumulative bit error count over the sixty frames is zero, the residual EVM is 3.49 percent with an effective-channel NMSE of 2.06e-3, the exact layer capacity is 43.5 bit/s/Hz, the measured per-layer post-equalization SINR is approximately 30 dB, and the CRC-gated throughput is 55.25 Mbit/s, the full payload rate of the accepted frames. The transmit-waveform Spectrum Analyzer of the passive branch shows the flat occupied band of the 144 active subcarriers with the steep CP-OFDM band edges.

6. Verification Methodology

This section defines the acceptance rule for the Simulink campaign. A result is accepted only when the active profile matches the model dimensions, the pre-simulation gates pass, the complete SNR grid is executed, and every aggregate remains within the declared tolerance of the MATLAB reference.

6.1 Pre-simulation verification gates

Table 3 describes the four gates executed by run_verification_gates.m in the model InitFcn. The first column identifies the gate, the second states the tested function, and the third defines the acceptance condition. Because the gates run before the first simulation step, a failed basic operation prevents the model from producing a numerical campaign.

Table 3. Mandatory verification gates executed before the first model step.

Gate	Verification target	Acceptance condition
Gate 1	Constellation normalization	Mean symbol power equals one for QPSK, 16-QAM, 64-QAM, and 256-QAM
Gate 2	Noiseless mapping round trip	Mapped and hard-demapped bits are identical
Gate 3	OFDM modulation/demodulation	Maximum reconstruction error is below 1e-12
Gate 4	CRC24A accept/reject	Valid block is accepted and a one-bit corruption is rejected

Gate 3 verifies the unitary transform pair used by ofdm_modulate.m and ofdm_demodulate.m. The executed project records a reconstruction error at double-precision level. Gate 4 verifies both sides of CRC behavior; checking only acceptance of an intact block would not prove that corruption is detected.

6.2 Reference-comparison protocol

Figure 3 shows the Simulink verification workflow. After the profile and fixed dimensions are checked, the gates are executed. Each SNR point is then simulated for 60 frames, the metrics are aggregated, and the result is compared with the MATLAB reference. A failed metric is not silently averaged into the campaign; it directs the workflow back to model, configuration, or statistical investigation

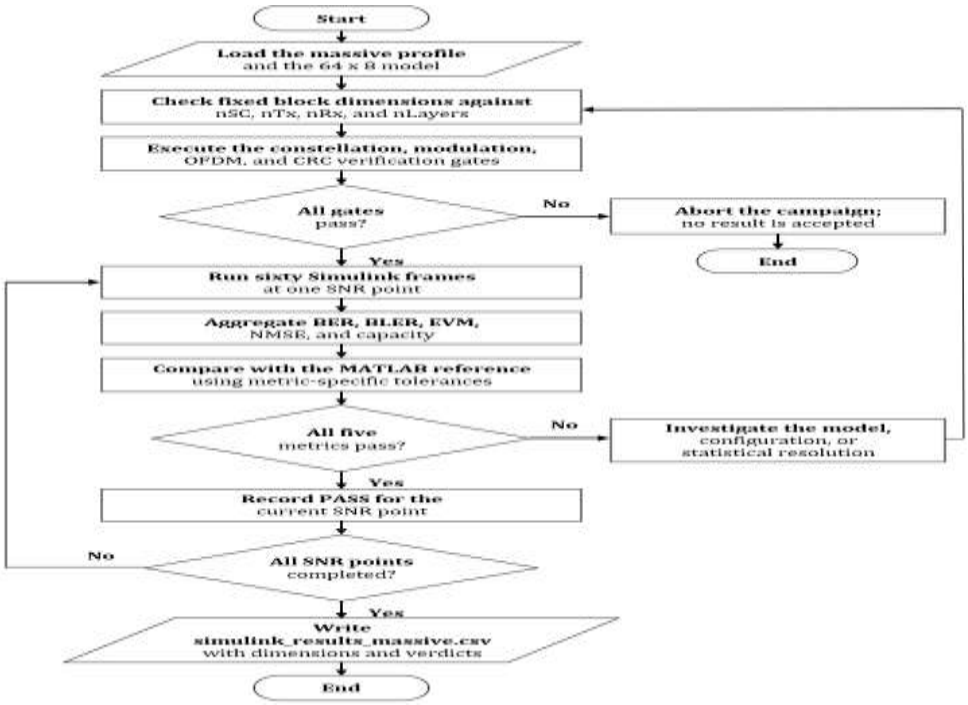

Figure 3. Gate-controlled Simulink SNR-sweep and MATLAB-reference verification workflow.

Table 4 lists the tolerances implemented in sweep_snr_grid.m. Error-rate quantities are compared in logarithmic decades because the relevant differences are multiplicative. EVM and capacity are compared by relative error, whereas BLER is compared by absolute difference because the 60-frame estimate is coarsely quantized in steps of 1/60.

Table 4. Metric-specific tolerances used by sweep_snr_grid.m.

Metric	Deviation rule	Acceptance tolerance
BER	Absolute log10 difference	0.3 decades
BLER	Absolute difference	0.25
EVM	Relative difference	10%
NMSE	Absolute log10 difference	0.15 decades
Capacity	Relative difference	2%

The sweep script also reads the MATLAB Function script inside the MIMO Channel Generator block and confirms that its preallocated channel dimensions match 144 x 8 x 64. This check prevents a compact model from being executed under the massive profile or a massive model from being interpreted as the compact profile.

6.3 Synchronization verification gates

Four deterministic gates verify the synchronization functions on the massive flat-Rayleigh profile before any Simulink execution, mirrored in MATLAB by run_sync_verification and executed on the Python reference implementation by advanced_sync_verification.py. Gate S1 passes the clean time-domain path with no impairment and requires an exact zero timing estimate, a negligible frequency estimate, and zero bit errors; the executed run returns a timing estimate of 0, a frequency estimate of -1.4e-13 subcarrier spacings, and 0 errors in 27,624 bits, which proves that the CP-OFDM waveform detour is numerically equivalent to the verified frequency-domain chain. Gate S2 injects the 7-sample offset and the 0.15-subcarrier frequency offset without noise and requires exact recovery; the executed run returns a timing estimate of exactly 7, a frequency estimate of 0.15000, and 0 errors. Gate S3 repeats the impaired point at 20 dB over twenty frames and measures a 20 of 20 timing hit rate, a mean absolute residual frequency error of 1.35e-4 subcarrier spacings, and a corrected bit error rate identical to the ideal-synchronization reference on matched seeds. Gate S4 leaves the same impairments uncorrected and records a bit error rate of 0.4905, which demonstrates the complete link failure that the synchronizer removes. The gate decisions and the measured estimates are written to advanced_sync_verification.csv next to the other result tables. Two further gates cover the extended impairment models. Gate S5 injects a 7.4-sample offset with a 0.10-subcarrier frequency offset and requires exact recovery of the integer part with zero bit errors; the executed run returns a timing estimate of exactly 7, a frequency estimate of 0.09988, and 0 errors in 27,624 bits, confirming that the 0.4-sample fractional residual is absorbed by the interpolating least-squares estimator. Gate S6 enables the Wiener phase-noise model: with a 10 Hz linewidth and no receiver noise the chain remains error-free, and with a 200 Hz linewidth at 20 dB together with the standard offsets the measured bit error rate is 4.086e-2, which quantifies the uncompensated symbol-to-symbol phase walk and is recorded as an informational row of the gate table.

7. Executed Results and Technical Analysis

This section reports the stored seven-point Simulink campaign, compares it with the MATLAB reference, discusses the error-rate and estimation trends, and presents the waveform-domain measurements obtained from the passive monitoring branch.

7.1 MATLAB-Simulink BER agreement

Figure 4 illustrates the BER produced by the MATLAB reference and the Simulink testbench. Both curves use the same 64 x 8 profile, four layers, 16-QAM, flat Rayleigh fading, wideband SVD precoding, LS estimation, unbiased-MMSE equalization, SNR grid, frame count, and random seed. The two executions are not expected to be bit-identical because the model scheduling and random-number consumption can differ; the acceptance criterion is statistical agreement under the tolerance of Table 4.

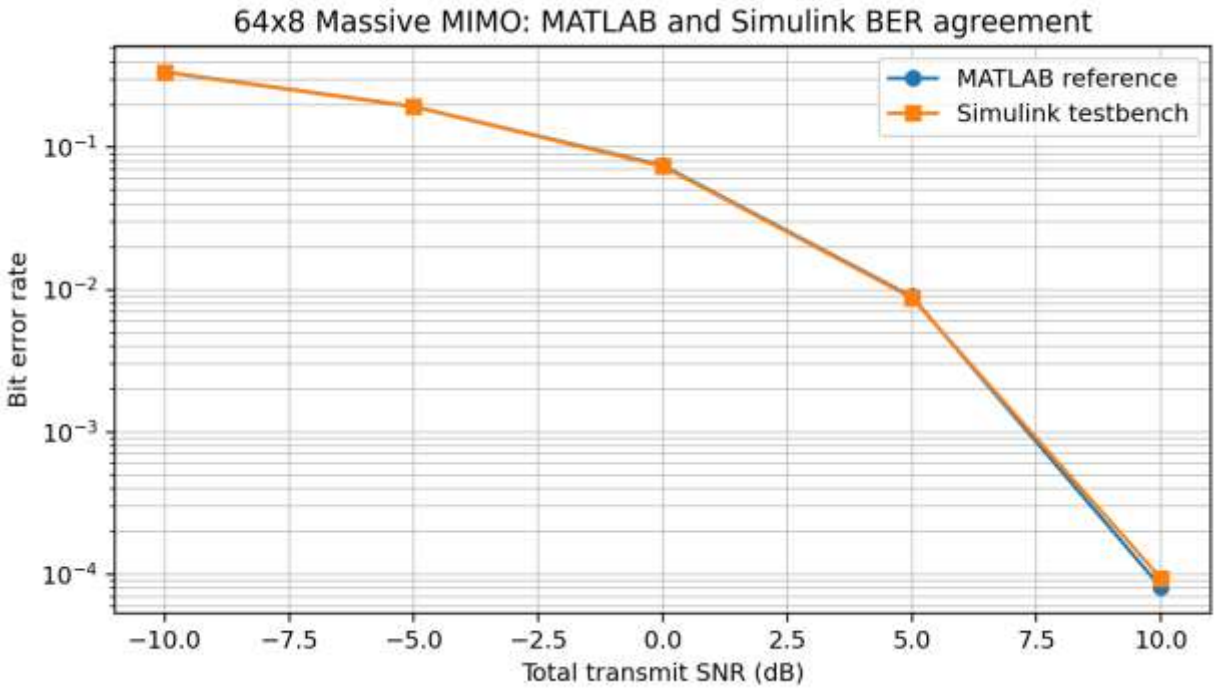

Figure 4. Original-color BER comparison generated from the stored MATLAB and Simulink CSV results. The zero-error observations at 15 and 20 dB are not plotted on the logarithmic axis.

The curves overlap throughout the resolved BER region. The largest difference occurs at 10 dB, where the Simulink BER is 9.35 x 10^-5 and the MATLAB reference is 8.08 x 10^-5, corresponding to 0.063 decades. This is well inside the 0.3-decade tolerance. The 15 and 20 dB points contain no observed bit errors in either 60-frame run; those points are finite-run observations and are not plotted as positive BER values on the logarithmic axis.

7.2 Complete metric results

Table 5 reports the Simulink values and verdicts stored in simulink_results_massive.csv. The first column gives total transmit SNR, the next five columns give the aggregate performance metrics, and the final column records the comparison verdict. The self-identifying CSV additionally stores nTx=64, nRx=8, and nLayers=4 in every row.

SNR (dB)	BER	BLER	EVM (%)	NMSE	Capacity	Verdict
-10	3.373e-01	1.00	84.59	1.390e+00	6.46	PASS
-5	1.937e-01	1.00	55.26	4.395e-01	11.64	PASS
0	7.345e-02	1.00	33.30	1.390e-01	17.72	PASS
5	8.748e-03	1.00	19.26	4.395e-02	24.18	PASS
10	9.352e-05	0.85	10.93	1.390e-02	30.76	PASS

SNR (dB)	BER	BLER	EVM (%)	NMSE	Capacity	Verdict
15	0.000e+00	0.00	6.17	4.395e-03	37.39	PASS
20	0.000e+00	0.00	3.47	1.390e-03	44.02	PASS

Table 5. Executed 64 x 8 Simulink sweep and stored comparison verdicts.

All seven operating points pass. Across the complete grid, the largest BER deviation is 0.063 decades, the largest EVM deviation is 0.63% relative, the largest NMSE deviation is 0.006 decades, and the largest capacity deviation is 0.47% relative. BLER is identical to the MATLAB reference at every stored point. These deviations are substantially smaller than the declared limits in Table 4.

7.3 Interpretation of the trends

BER falls from approximately 0.337 at -10 dB to 9.35 x 10^-5 at 10 dB. BLER remains one through 5 dB because a CRC-protected block contains 27,624 payload bits; even a low residual BER makes at least one block error probable. At 10 dB the BLER falls to 0.85, and the 15 and 20 dB runs observe no block errors. EVM decreases monotonically from 84.6% to 3.47%, indicating progressive contraction of the equalized 16-QAM clusters. NMSE decreases from 1.39 to 1.39 x 10^-3, approximately one decade per 10 dB, which is consistent with an LS estimator dominated by pilot noise. Capacity increases from 6.46 to 44.02 bit/s/Hz as the received SNR increases.

7.4 RF-oriented waveform measurements

Figure 5 shows the live Spectrum Analyzer and Constellation Diagram attached to the passive monitoring branch. The spectrum view displays the 4.32 MHz occupied allocation within the 7.68 MHz sampled band, while the constellation view shows the equalized four-layer 16-QAM symbols. These scopes are diagnostic instruments; the reproducible numerical figures are generated from the workspace logs by plot_rf_measurements.m.

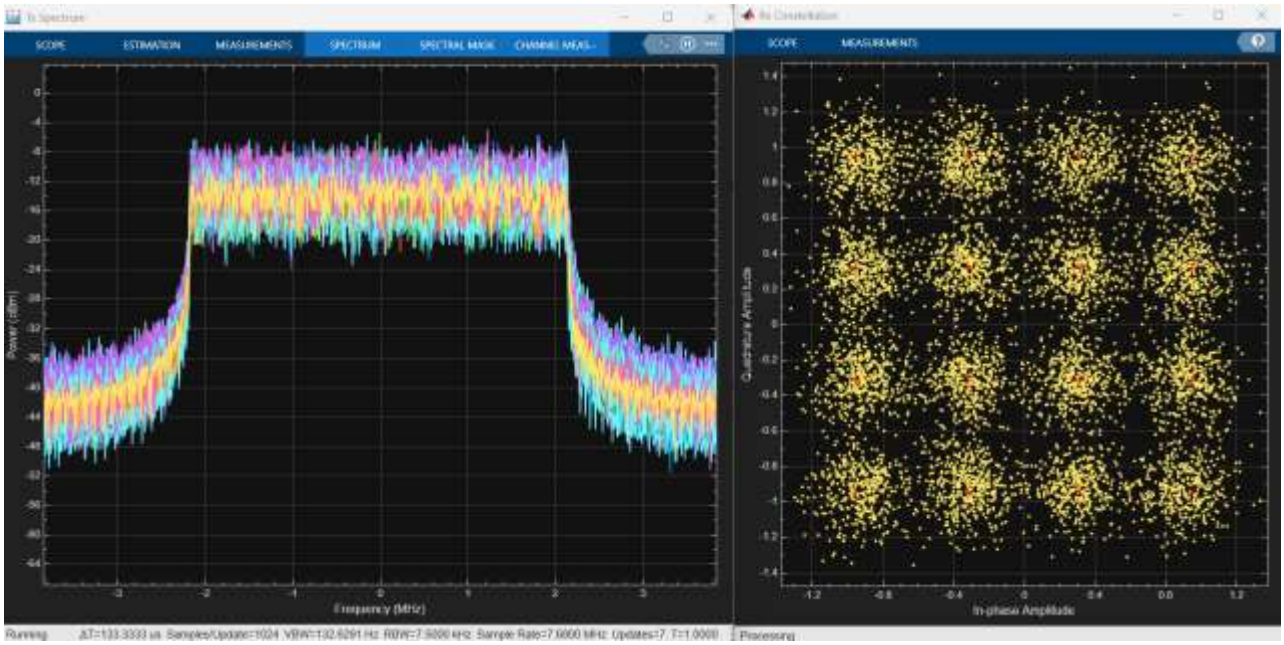

Figure 5. Original live Spectrum Analyzer and Constellation Diagram capture from the instrumented Simulink model.

Figure 6 presents the three original RF measurement files generated by plot_rf_measurements.m. They are inserted separately at full readable width rather than converted to grayscale or compressed into a small composite. Figure 6(a) is the Hann-windowed Welch PSD of the continuous CP-OFDM waveform. The dashed vertical markers identify the nominal occupied-band edges of the 144-subcarrier allocation within the 7.68 MHz sampled band.

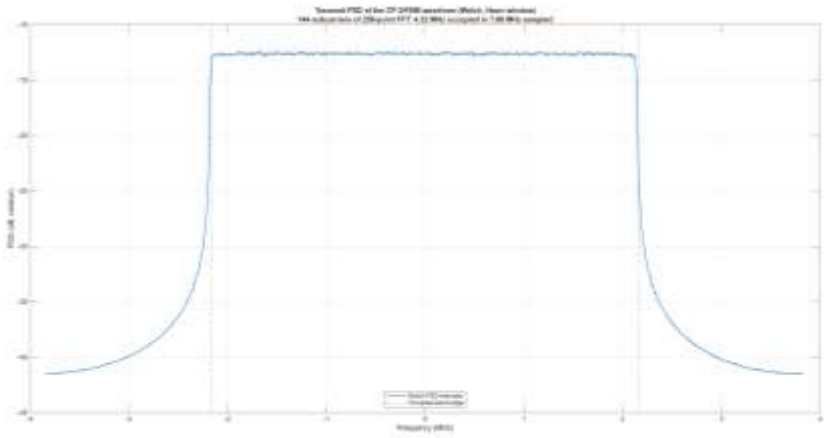

Figure 6(a). Original transmit PSD of the CP-OFDM waveform.

Figure 6(b) shows the complementary cumulative distribution of PAPR over the 14 OFDM symbols, 64 transmit antennas, and 60 simulated frames. The curve quantifies the probability that a per-antenna OFDM symbol exceeds a selected PAPR threshold and provides the appropriate starting point for later power-amplifier back-off and nonlinearity studies.

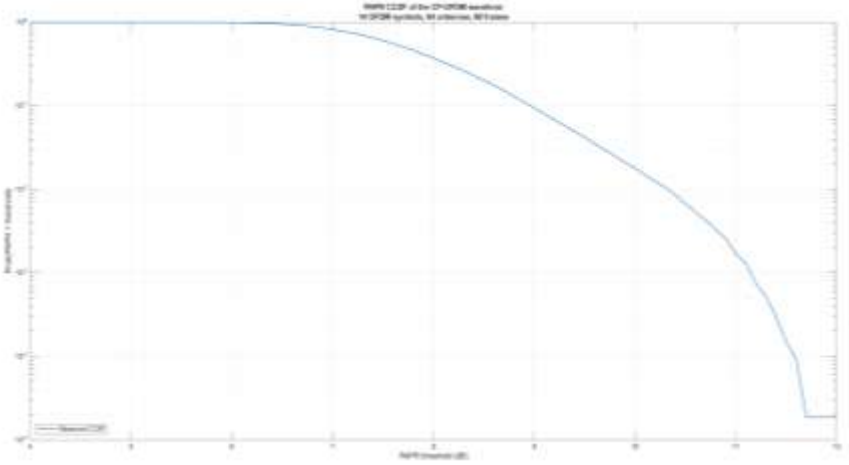

Figure 6(b). Original PAPR CCDF of the transmitted CP-OFDM waveform.

Figure 6(c) shows the original equalized 16-QAM constellation with the four spatial layers overlaid. The location of the clusters relative to the ideal reference points confirms that the unbiased-MMSE normalization preserves the amplitude-bearing 16-QAM decision geometry, while the remaining scatter is consistent with the measured EVM at the captured operating point.

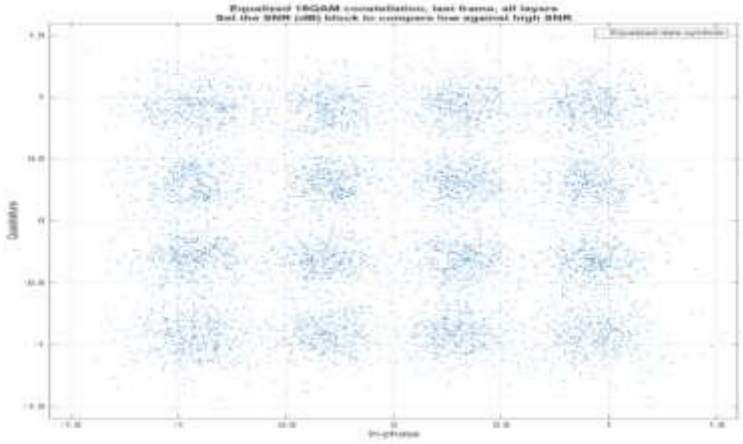

Figure 6(c). Original equalized 16-QAM constellation generated from the Simulink workspace log.

The PSD confirms the expected occupied bandwidth and rectangular-pulse CP-OFDM sidelobes [5]. The PAPR distribution extends into the approximately 11-12 dB range in the supplied run, indicating the transmitter headroom that would be required before adding a nonlinear power-amplifier model. The constellation clusters remain centered on the normalized 16-QAM reference points because the diagonal bias correction is applied after MMSE equalization.

7.5 Executed advanced-testbench results

The advanced model was executed on the massive profile with the standard impairment operating point. At 20 dB the complete sixty-frame run records zero bit errors, an EVM of 3.49 percent, an NMSE of 2.06e-3, a capacity of 43.5 bit/s/Hz, a per-layer SINR near 30 dB, and a throughput of 55.25 Mbit/s, with the synchronizer reporting the injected impairments exactly. At the -10 dB end of the sweep the same model records a cumulative BER of 0.344, a block error rate of one, an EVM of 85 percent, and zero throughput, while the timing estimate still reads 7 samples and the frequency estimate 0.147; the cyclic-prefix metric remains reliable at this operating point because it accumulates over fourteen symbols and eight receive antennas, and only the frequency estimate broadens with the noise. Both gate sets executed before every run of the campaign: the four chain gates and the three MATLAB synchronization gates, whose measured values, a clean-path timing estimate of 0 with a residual frequency error of -2.7e-14, exact recovery of the 7-sample and 0.15-subcarrier impairments, and exact integer-part recovery of the fractional 7.4-sample case with zero errors in all three gates, reproduce the executed Python reference gates on independent MATLAB numerics.

8. Reproduction Procedure and Project Contents

This section describes the complete reproduction sequence, the role of the principal project files, the expected output artifacts, and the checks that should be performed before the results are uploaded or cited. The procedure is designed so that a new user can rebuild the model rather than depending only on the supplied SLX file.

8.1 Reproduction workflow

Figure 7 illustrates the recommended reproduction workflow. The workflow begins in the Matlab_File directory because config.m resolves the massive reference path relative to that location. The MATLAB reference is generated or confirmed before the Simulink sweep, the model is rebuilt for the active profile, the complete SNR grid is executed, and the result CSV and RF figures are inspected before the package is archived.

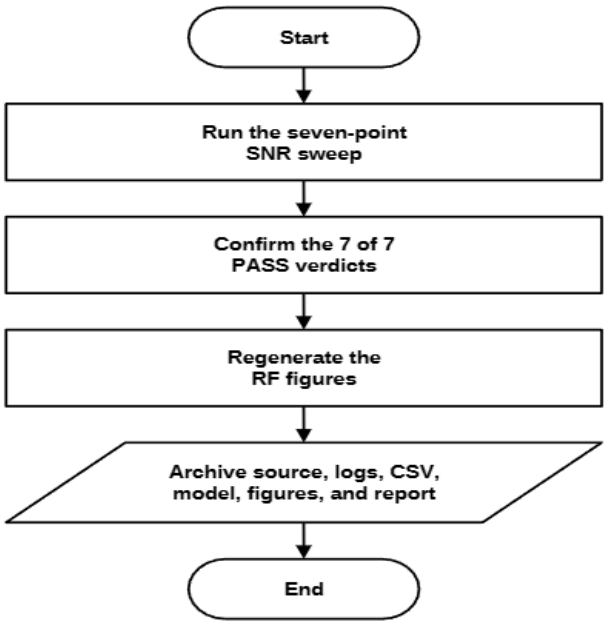

Figure 7. Reproduction and validation workflow for the 64 x 8 Simulink testbench.

Algorithm 2: Reproduce the Simulink campaign and RF measurements
1: Set the MATLAB Current Folder to 01_5G_NR_Massive_MIMO_Simulink_Testbench/Matlab_File and add the project folder to the path.
2: Run set_sim_profile(massive) so config.m returns 64 transmit antennas, 8 receive antennas, four layers, the massive SNR grid, and 60 frames per point.
3: Run run_massive_mimo to regenerate matlab_results_massive.csv and its configuration log, or confirm that the supplied reference corresponds to the current code.
4: Run build_massive_testbench to back up the compact model, regenerate all fixed block dimensions, save NR_PDSCH_LinkLevel_Sim.slx, and add the RF-monitoring branch.
5: Open the model and confirm the dimension-derived title, stop time of 59, 64 x 8 channel interface, 64 x 4 precoder interface, and four-layer receiver interface.
6: Run sweep_snr_grid. Confirm that all seven SNR points report PASS and that simulink_results_massive.csv contains the correct nTx, nRx, and nLayers columns.
7: Run the instrumented model at the desired SNR and then execute plot_rf_measurements to regenerate rf_psd.png, rf_papr_ccdf.png, and rf_constellation.png.
8: Archive the model, source functions, configuration log, MATLAB reference CSV, Simulink CSV, figures, README, report, license, and citation metadata.

The exact aggregate BER should be calculated from the workspace arrays rather than copied from the live cumulative display because the denominator integrator starts from one to avoid division by zero at the first step. The correct expression is sum(log_bitErrors)/sum(log_nBits).

8.2 Project file organization

Table 6 describes the principal files and folders needed to reproduce the report. The first column identifies the artifact, the second states its role, and the third explains when it is created or used. The table distinguishes source files from generated results so version control can retain the reproducible artifacts while excluding Simulink caches.

Table 6. Principal reproducibility files and their roles.

Artifact	Purpose	Execution role
README.md	Project overview, requirements, execution commands, results, and limitations	Read before execution
Matlab_File/config.m	Single source of numerology, MIMO dimensions, channel, receiver, seed, grid, and outputs	Loaded by every wrapper
Matlab_File/build_massive_testbench.m	Selects the massive profile and rebuilds the model	Run after dimension changes
Matlab_File/NR_PDSCH_LinkLevel_Sim.slx	Executable graphical testbench	Built and then executed
Matlab_File/sl_*.m	Thin interfaces between Simulink and the core MATLAB functions	Called by MATLAB Function blocks

Artifact	Purpose	Execution role
Matlab_File/sweep_snr_grid.m	Seven-point sweep, metric comparison, and CSV generation	Run after the model is built
04_Simulation_Results/MATLAB/csv	MATLAB reference CSV and configuration log	Generated by run_massive_mimo
Matlab_File/simulink_results_massive.csv	Self-identifying Simulink result artifact	Generated by sweep_snr_grid
Figures/ and Report/	Model images, plots, workflow diagrams, DOCX, and PDF	Used for documentation

Generated folders such as slprj, codegen, autosave files, and platform-specific MEX outputs are excluded by .gitignore because they are not source artifacts. The supplied project retains the SLX model and the executed CSV and PNG results so readers can inspect the completed campaign without rebuilding, while the builder scripts allow the fixed interfaces to be regenerated from the source configuration.

The advanced testbench adds the following files to the package: build_advanced_testbench.m constructs the hierarchical advanced model, sl_waveform_tx.m, sl_channel_impairments.m, sl_synchronizer.m, and sl_link_quality.m implement the time-domain path, the impairment injection, the cyclic-prefix synchronizer, and the SINR and throughput measurement, run_sync_verification.m executes the deterministic synchronization gates in MATLAB, run_advanced_sweep.m executes the model across the configured SNR grid and writes simulink_advanced_sweep.csv, and advanced_sync_verification.py together with results/advanced_sync_verification.csv provides the executed Python verification of the same algorithms.

9. Limitations, Future Work, and Conclusion

This section states the limits of the current testbench so that the results are interpreted only within the executed assumptions. The model represents a single-cell, single-user digital-baseband downlink in which the baseline bench assumes ideal synchronization, while the advanced bench injects and corrects an integer timing offset and a fractional carrier frequency offset; fractional-sample

timing errors, phase noise, and time-varying Doppler tracking remain unmodeled. The massive profile uses a flat independent Rayleigh channel without spatial correlation, explicit array geometry, Doppler, delayed CSI, or codebook quantization. The transmitter has ideal channel knowledge for wideband precoder construction, and the Simulink transport chain is uncoded except for CRC24A protection.

The Simulink blocks call the MATLAB algorithms through extrinsic wrappers; therefore, the model is a third execution environment and integration testbench, not an independently rewritten numerical reference. The fixed 60-frame population provides useful regression resolution but is insufficient for statistically tight high-SNR error-rate statements. Future work should include adaptive stopping with raw error counts and confidence intervals, paired common-input checkpoints, TDL-A and TDL-C channels, spatially correlated ULA or UPA arrays, low and moderate Doppler, LMMSE channel smoothing, delayed and quantized CSI, LDPC-coded transport processing, and code-generation-compatible subsystem implementations. RF extensions should add phase noise, IQ imbalance, PA nonlinearity, DPD, and ACLR/EVM analysis starting from the measured PSD and PAPR.

One implementation detail requires explicit control when extending the project: the uploaded estimator performs linear edge extrapolation. On a frequency-selective channel, this choice can amplify pilot noise at the allocation edges. A nearest-pilot extension, Wiener interpolation, or explicit edge-pilot design should be evaluated and locked before using the testbench for publication-level frequency-selective comparisons [8]-[10].

9.1 Conclusion

This report documented a technically traceable Simulink testbench for a PDSCH-oriented 5G NR 64 x 8 Massive MIMO-OFDM link. The mathematical equations in Sections 2 and 4 are the implementation-independent physical-layer and receiver specifications, not formulas created specifically for Simulink. The model is generated programmatically from one locked configuration, executes one complete four-layer Monte Carlo frame per fixed step, and connects transmitter, channel, wideband SVD precoding, LS effective-channel estimation, unbiased-MMSE equalization, CRC detection, metric extraction, workspace logging, and RF-oriented instrumentation. Every Simulink subsystem is mapped to a named wrapper and a core MATLAB implementation, so the mathematical specification, graphical model, and source code remain aligned.

The verification design prevents basic numerical defects from entering a campaign: constellation, modulation, OFDM, and CRC gates run before the first frame; the sweep script verifies the fixed model dimensions; and each SNR point is compared with the MATLAB reference under metric-specific tolerances. The stored 64 x 8 campaign passes at all seven SNR points. The largest BER deviation is 0.063 decades, the EVM, NMSE, and capacity differences remain far below their limits, and BLER matches the reference at every point. The passive waveform branch provides spectrum, PAPR, and constellation evidence without changing the communication metrics. The resulting package is therefore suitable as a focused portfolio project in MATLAB-Simulink system modeling and testbench validation.

10. References

[1] 3GPP TS 38.211, NR; Physical channels and modulation.

[2] 3GPP TS 38.212, NR; Multiplexing and channel coding.

[3] 3GPP TS 38.214, NR; Physical layer procedures for data.

[4] 3GPP TR 38.901, Study on channel model for frequencies from 0.5 to 100 GHz.

[5] J. G. Proakis and M. Salehi, Digital Communications, 5th ed., McGraw-Hill, 2008.

[6] D. Tse and P. Viswanath, Fundamentals of Wireless Communication, Cambridge University Press, 2005.

[7] E. Telatar, "Capacity of multi-antenna Gaussian channels," European Transactions on Telecommunications, vol. 10, no. 6, pp. 585-595, 1999.

[8] J.-J. van de Beek et al., "On channel estimation in OFDM systems," Proc. IEEE VTC, 1995, pp. 815-819.

[9] O. Edfors et al., "OFDM channel estimation by singular value decomposition," IEEE Transactions on Communications, vol. 46, no. 7, pp. 931-939, 1998.

[10] S. Coleri et al., "Channel estimation techniques based on pilot arrangement in OFDM systems," IEEE Transactions on Broadcasting, vol. 48, no. 3, pp. 223-229, 2002.

[11] MathWorks, Simulink Documentation, The MathWorks, Inc.

[12] MathWorks, 5G Toolbox Documentation, The MathWorks, Inc.